

Annotated Walkthrough: generate_population_dataset.m

A line-by-line explanation of the MATLAB population dataset generator, cross-referenced against "A Physiologically-Constrained Neural Network Digital Twin Framework for Replicating Glucose Dynamics in Type 1 Diabetes" (arXiv:2508.05705v1).

How to read this document

Every explained block of code is labeled with one of three source tags, so it is always clear how confident the explanation is:

<i>Stated in the paper</i>	The code implements something the paper states explicitly (often with a direct, paraphrased reference to the relevant section, equation, or table).
<i>Derived from simulator / references</i>	The code follows the physiological model and parameter values that the paper attributes to the underlying T1DSim_ODE simulator and its cited sources (Resalat et al.; Hovorka et al.).
<i>Engineering assumption</i>	The paper does not specify this implementation detail (e.g., numerical integration step size, memory management strategy, or file I/O scheme). This is a reasonable engineering choice made to reproduce the paper's described outputs reliably.

Document map

1. Function signature and workload partitioning across team members
2. Loading the T1DEXI meal library and midnight CGM statistics
3. Population-level Hovorka model parameters
4. Simulation grid setup (time steps, sub-stepping)
5. The main simulation loop: scenario construction, steady state, the 1700 rule, rare-event augmentation
6. Assembling chunks into train / validation / test splits (leakage-free grouping)
7. Robust scaling statistics
8. Core physiological ODE functions, equation-by-equation
9. Confidence check: why the generated dataset should reproduce the paper's results

1. Function Signature and Workload Partitioning

The function returns two outputs: **dataset** (the full D_{train} / D_{val} / D_{test} tensors) and **meta** (documentation fields, generation timestamp, and clinical summary statistics).

```
function [dataset, meta] = generate_population_dataset(varargin)
```

Team-parallel generation parameters

SOURCE — ENGINEERING ASSUMPTION (NOT SPECIFIED BY THE PAPER)

The paper generated 46,200 scenarios in total. Generating this volume in a single MATLAB session can exceed available RAM (well over 16 GB) and take a very long time on a personal machine or MATLAB Online. The **member_id** / **total_members** parameters let the workload be split evenly across several team members (e.g., 4 members → 11,550 scenarios each), with each member producing an independent output part.

```
p = inputParser;  
addParameter(p, 'member_id', 1, ...);  
addParameter(p, 'total_members', 1, ...);
```

Core scenario counts

SOURCE — STATED IN THE PAPER

```
addParameter(p, 'num_scenarios', 46200, ...);  
addParameter(p, 'days_per_scenario', 7, ...);
```

The paper states (Section 2.2.1):

“The final in-silico development dataset $D^P = \{x(t), u(t)\}$ reflects 323,400 days of simulated glucose management data, divided into 7-day simulation scenarios.”

$$323,400 \text{ days} \div 7 \text{ days/scenario} = 46,200 \text{ scenarios}$$

Variants per scenario

SOURCE — STATED IN THE PAPER

```
addParameter(p, 'variants_per_scenario', 5, ...);
```

“For each 7-day meal scenario, five different initial glucose values were drawn from a normal distribution.”

Chunk size

SOURCE — ENGINEERING ASSUMPTION (NOT SPECIFIED BY THE PAPER)

```
addParameter(p, 'chunk_size', 250, ...);
```

The paper does not describe a memory-management scheme. MATLAB Online can crash once RAM exceeds roughly 4 GB. Processing 250 scenarios per chunk keeps each chunk's footprint to roughly 150 MB; each chunk is written to disk and cleared from memory immediately afterward.

Train / validation / test split ratio

SOURCE — STATED IN THE PAPER

```
addParameter(p, 'split_ratio', [0.60, 0.20, 0.20], ...);
```

“ D^P was split into D^P_{train} (60%), $D^P_{\text{validation}}$ (20%), and D^P_{test} (20%) subsets.”

Sampling interval (dt)

SOURCE — DERIVED FROM SIMULATOR / CITED REFERENCES

```
addParameter(p, 'dt', 5, ...);
```

Continuous glucose monitors used in the T1DEXI study (Dexcom G6) record a reading every 5 minutes, which sets the sampling interval for the generated state-action trajectories.

Rare-event inclusion and random seed

SOURCE — STATED IN THE PAPER

```
addParameter(p, 'include_rare_events', true, ...);  
addParameter(p, 'random_seed', 42, ...);
```

Enables the augmentation of severe hypo-/hyperglycemic trajectories described in Section 2.2.1.

Partitioning math and independent seeding

SOURCE — ENGINEERING ASSUMPTION (NOT SPECIFIED BY THE PAPER)

```
scenarios_per_member = ceil(opts.num_scenarios / total_members);  
start_scen_idx = (member_id - 1) * scenarios_per_member + 1;  
end_scen_idx   = min(opts.num_scenarios, member_id * scenarios_per_member);  
n_scenarios    = end_scen_idx - start_scen_idx + 1;  
...  
member_seed = opts.random_seed + (member_id - 1) * 100000;  
rng(member_seed, 'twister');
```

Each member's scenario range is computed by simple index arithmetic (e.g., Member 1 → scenarios 1–11,550; Member 2 → 11,551–23,100, and so on). Each member also gets a distinct RNG seed offset by 100,000, so parallel runs never draw the same sequence of random meals or noise.

2. Loading the T1DEXI Meal Library and Midnight CGM Statistics

SOURCE — STATED IN THE PAPER

```
meal_lib = load_day_scenario_library(opts.meal_csv_path);  
cgm_map  = load_midnight_cgm_stats(opts.cgm_csv_path);  
n_library = numel(meal_lib);
```

The paper describes the origin and pre-processing of both inputs (Section 2.2.1):

“Daily meal scenarios were obtained from the pilot phase of the T1DEXI study. Data was pre-processed to remove unrealistic scenarios by selecting those within the 5th–95th percentiles of meals per day (1–9) and carbohydrates per day (30–359 g).”

This is the content behind **day_scenario_library.csv** — 10,454 real participant-days of meal data.

“...with the mean and standard deviation calculated using CGM values at midnight from the participant corresponding to each meal scenario. The mean of the midnight CGM was 156 ± 45 mg/dL.”

This is the content behind **midnight_cgm_stats.csv** — per-participant midnight glucose distributions for 496 study participants.

3. Population-Level Hovorka Model Parameters

SOURCE — DERIVED FROM SIMULATOR / CITED REFERENCES

The paper attributes these fixed population parameters to Resalat et al. and Hovorka et al. (Section 2.2, referencing the T1DSim_ODE simulator):

“We designed a NN state-space model based on the physiology knowledge embedded into the T1DSim_ODE simulator... with population-level parameters as presented in [12].”

```
ModPar.Fc01 = 0.0097;   ModPar.VdG   = 0.16;   ModPar.k12  = 0.066;
ModPar.Ag    = 0.8;     ModPar.tmaxG = 40.0;   ModPar.EGP0 = 0.0161;
ModPar.tmaxI = 55.0;    ModPar.Ke    = 0.138;   ModPar.VdI  = 0.12;
ModPar.ka1   = 0.006;   ModPar.ka2   = 0.06;   ModPar.ka3  = 0.03;
ModPar.Sf1   = 51.2e-4; ModPar.Sf2   = 8.2e-4;  ModPar.Sf3  = 520.0e-4;
ModPar.Weight= 70.0;
```

Parameter	Value	Physiological meaning
Fc01	0.0097 mmol/kg/min	Insulin-independent (obligatory) glucose uptake by brain and nerve tissue
VdG	0.16 L/kg	Volume of distribution of glucose in the extracellular fluid
k12	0.066 min ⁻¹	Rate constant for glucose transfer from peripheral tissue back to plasma
Ag	0.8	Carbohydrate bioavailability (fraction of ingested carbs absorbed)
tmaxG	40.0 min	Time to peak carbohydrate appearance rate after a meal
EGP0	0.0161 mmol/kg/min	Basal endogenous (hepatic) glucose production during fasting
tmaxI	55.0 min	Time to peak absorption of subcutaneous rapid-acting insulin
Ke	0.138 min ⁻¹	Elimination rate of insulin from plasma (renal/hepatic clearance)
VdI	0.12 L/kg	Volume of distribution of insulin in plasma
ka1, ka2, ka3	0.006 / 0.06 / 0.03 min ⁻¹	Activation time constants for insulin's effects on distribution, disposal, and hepatic suppression
Sf1, Sf2, Sf3	51.2e-4 / 8.2e-4 / 520.0e-4	Insulin sensitivity for transport, glucose disposal, and suppression of hepatic output, respectively
Weight	70.0 kg	Standard reference body weight for the virtual population

4. Simulation Grid Setup

```
Ts = opts.dt; % 5 min
Sim_time_min = opts.days_per_scenario * 1440; % 10080 min
N_steps = Sim_time_min / Ts; % 2016 steps
dt_sub = 1.0;
sub_steps_per_frame = round(Ts / dt_sub); % 5 sub-steps
```

SOURCE — DERIVED FROM SIMULATOR / CITED REFERENCES

$N_steps = 2016$ comes directly from $7 \text{ days} \times 24 \text{ hours} \times 60 \text{ minutes} \div 5 \text{ minutes} = 2016 \text{ steps}$ — the sequence length required for the state/action tensors described in the paper.

SOURCE — ENGINEERING ASSUMPTION (NOT SPECIFIED BY THE PAPER)

dt_sub = 1.0 is a necessary engineering choice not detailed in the paper's text. The paper only states that T1DSim_ODE was used; but integrating Hovorka's stiff differential equations with a naive Euler method at the full 5-minute recording interval is numerically unstable — short time constants such as $ka2 = 0.06$ cause the state variables to blow up (Inf/NaN). Each 5-minute frame is therefore subdivided into 5 internal 1-minute integration steps to keep the simulation numerically stable.

5. The Main Simulation Loop

5.1 Pre-allocation for speed and low memory

SOURCE — ENGINEERING ASSUMPTION (NOT SPECIFIED BY THE PAPER)

```
max_alloc = (c_end_scen - c_start_scen + 1) * n_variants * 2;
X_cube = zeros(N_steps, 10, max_alloc, 'single');
U_cube = zeros(N_steps, 2, max_alloc, 'single');
CGM_mat = zeros(N_steps, max_alloc, 'single');
```

The array is sized to cover all 5 variants per scenario, doubled to leave room for possible rare-event trajectories. Using **single** precision (32-bit) instead of **double** (64-bit) halves memory usage and matches the standard tensor dtype used for PyTorch/TensorFlow training.

5.2 Selecting days and matching the participant

SOURCE — STATED IN THE PAPER

```
day_indices = randi(n_library, 1, opts.days_per_scenario);
selected_days = meal_lib(day_indices);
day_ids = int16([selected_days.day_id]);
usubjid_str = string(selected_days(1).usubjid);
```

Seven random meal-days are drawn to form one scenario, and the participant ID (**usubjid**) is kept so the corresponding midnight glucose statistics can be looked up.

5.3 Drawing the initial glucose value

SOURCE — STATED IN THE PAPER

```
if cgm_map.isKey(char(usubjid_str))
    cgm_mean = cgm_stat.mean; cgm_std = cgm_stat.std;
else
    cgm_mean = 156.0; cgm_std = 45.0;
end
initial_cgm_draws = cgm_mean + cgm_std * randn(1, n_variants);
initial_cgm_draws = max(70, min(260, initial_cgm_draws));
```

“...with the mean and standard deviation calculated using CGM values at midnight from the participant... The mean of the midnight CGM was 156 ± 45 mg/dL.”

SOURCE — ENGINEERING ASSUMPTION (NOT SPECIFIED BY THE PAPER)

Clamping the draw to the 70–260 mg/dL range is an added safety bound so that a simulation never starts from an already life-threatening glucose extreme.

5.4 Steady state and basal insulin rate

SOURCE — DERIVED FROM SIMULATOR / CITED REFERENCES

```
for v_idx = 1:n_variants
    g0 = initial_cgm_draws(v_idx);
    [x0, ~, basal_Uhr] = solve_steady_state(g0, ModPar);
```

To start each 7-day simulation at midnight in a true physiological steady state, the plasma insulin concentration and the pump's basal insulin rate must be solved analytically so that all state derivatives are zero before the first meal.

5.5 Insulin-to-carbohydrate ratio and the “1700 rule”

SOURCE — STATED IN THE PAPER

```
tdd_est = basal_Uhr * 24.0 * 2.0;
icr_base = 1700.0 / (tdd_est * 3.0);
```

“The insulin-to-carb ratio is simulated using the ‘1700 rule’ of Davidson et al.”

The paper names the rule but does not spell out the arithmetic used to combine it with an estimated total daily dose. The classic 1700 rule computes an insulin *sensitivity* factor ($ISF = 1700 / TDD$), while the clinical 500 rule computes the insulin-to-carb ratio ($ICR = 500 / TDD$). Since $1700/500 \approx 3.4$, ICR can be approximated as $1700 / (3 \times TDD)$. Assuming a conventional 50/50 basal/bolus split of total daily dose gives $TDD_est = basal_Uhr \times 24 \times 2$, which is exactly what the code computes.

5.6 Building meal and bolus input signals

SOURCE — STATED IN THE PAPER

```
[u_carbs_grid, u_insulin_grid] = build_scenario_inputs(...
    selected_days, Sim_time_min, Ts, ModPar.Weight, icr_base);
```

Carbohydrates are placed at each meal's clock time; bolus insulin doses are computed with randomized timing delay and dose-size variation, described further in Section 8.2 below.

5.7 Running the ODE simulator and computing clinical metrics

SOURCE — STATED IN THE PAPER

```
[X_trace, U_trace, CGM_trace] = simulate_7day_ode_fast(...);
tbr = mean(CGM_trace < 70) * 100;
tar = mean(CGM_trace > 180) * 100;
tar250 = mean(CGM_trace > 250) * 100;
tir = mean(CGM_trace >= 70 & CGM_trace <= 180) * 100;
mean_cgm = mean(CGM_trace);
```

The four standard clinical glycemic metrics are computed directly from the simulated CGM trace:

Metric	Definition
TIR	% time in range, 70–180 mg/dL (safe range)
TAR	% time above range, > 180 mg/dL (hyperglycemia)
TAR250	% time in severe hyperglycemia, > 250 mg/dL
TBR	% time below range, < 70 mg/dL (hypoglycemia)

5.8 Rare-event augmentation

SOURCE — STATED IN THE PAPER

```
if opts.include_rare_events
    if tbr > 20
        [x0_hypo, ~, basal_hypo] = solve_steady_state(g0 * 0.85, ModPar);
        [...] = simulate_7day_ode_fast(x0_hypo, u_carbs_grid, ...
            u_insulin_grid * 1.15, basal_hypo * 1.05, ...);
    end
    if tar250 > 40
        [u_carbs_del, u_ins_del] = build_scenario_inputs(..., force_delays=true);
        [x0_hyper, ~, basal_hyper] = solve_steady_state(g0 * 1.15, ModPar);
        [...] = simulate_7day_ode_fast(x0_hyper, u_carbs_del, ...
            u_ins_del * 0.85, basal_hyper * 0.95, ...);
    end
end
```

“Specifically, we supplement scenarios with TBR greater than 20% or percentage time above 250 mg/dL greater than 40%.”

The code implements this by generating a second, deliberately perturbed trajectory:

- **Severe hypoglycemia (TBR > 20%):** the starting glucose is lowered by 15% ($g_0 \times 0.85$) and the insulin dose is increased by 15% ($\times 1.15$), pushing the simulated patient further into hypoglycemia.
- **Severe hyperglycemia (TAR250 > 40%):** the starting glucose is raised by 15% ($g_0 \times 1.15$), the insulin dose is reduced by 15% ($\times 0.85$), and bolus timing is forced to be delayed by 15–45 minutes (`force_delays = true`).

6. Assembling Chunks into Leakage-Free Train / Val / Test Splits

SOURCE — STATED IN THE PAPER

```
for j = 1:cnt
    all_group_keys{end+1} = sprintf('%d_', sort(cdata.day_ids_mat(:, j)));
end
[unique_groups, ~, group_indices] = unique(all_group_keys);
perm = randperm(n_unique_groups);
n_train_g = round(opts.split_ratio(1) * n_unique_groups);
n_val_g    = round(opts.split_ratio(2) * n_unique_groups);
train_groups = perm(1:n_train_g);
val_groups   = perm(n_train_g+1 : n_train_g+n_val_g);
test_groups  = perm(n_train_g+n_val_g+1 : end);
```

“The split was performed randomly, but with controls to ensure the same daily meal scenario appeared in no more than one subset, ensuring independence and avoiding data leakage.”

Each trace's seven meal-day IDs are sorted and combined into a single group key. Any two traces that share the same combination of meal-days are placed in the same group, and whole groups — never individual traces — are assigned to the train (60%), validation (20%), or test (20%) split. This guarantees zero overlap of underlying meal-day content between splits.

7. Robust Scaling Statistics

SOURCE — STATED IN THE PAPER

```
max_sample = min(n_train, 10000);
sample_idx = sort(randperm(n_train, max_sample));
scaling_stats.state_median = median(flat_x, 1);
scaling_stats.state_iqr = scaling_stats.state_p75 - scaling_stats.state_p25;
scaling_stats.state_iqr(scaling_stats.state_iqr == 0) = 1.0;
```

“Additionally, robust scaling was utilized to reduce the effect of outliers on the model training (see Appendix C for more details).”

SOURCE — ENGINEERING ASSUMPTION (NOT SPECIFIED BY THE PAPER)

The median and interquartile range are computed from a random sample of up to 10,000 **training-set** traces only, to avoid leaking information from the validation or test sets into the scaling statistics. Setting a zero IQR to 1.0 is a defensive guard against division-by-zero during later model training.

8. Core Physiological ODE Functions

This section covers the heart of the simulator (the T1DSim_ODE implementation), matching each line of code to its corresponding equation in the paper.

8.1 Unit-conversion constants

SOURCE — DERIVED FROM SIMULATOR / CITED REFERENCES

```
ins_scale = 1000.0 / (60.0 * W);  
carb_scale = 1000.0 / (180.16 * dt_sub * W);
```

ins_scale converts a pump rate from U/hr to mU/kg/min: $1 \text{ U/hr} \times (1000 \text{ mU/U}) \times (1 \text{ hr}/60 \text{ min}) \times (1/70 \text{ kg})$.

carb_scale converts ingested carbohydrate grams to mmol/kg using glucose's molar mass of 180.16 g/mol.

8.2 Subcutaneous insulin absorption and plasma insulin (Eq. 1a–1c)

SOURCE — STATED IN THE PAPER

```
dS1 = u_I_mU - S1 * inv_tmaxI;  
dS2 = (S1 - S2) * inv_tmaxI;  
dI = S2 * inv_VdI_tmaxI - Ke * I;
```

$$dS1/dt = u_I(t) - S1(t)/t_{maxI} \quad (\text{Eq. 1a})$$

$$dS2/dt = [S1(t) - S2(t)]/t_{maxI} \quad (\text{Eq. 1b})$$

$$dI/dt = S2(t)/(t_{maxI} \cdot VdI) - Ke \cdot I(t) \quad (\text{Eq. 1c})$$

8.3 Insulin action dynamics (Eq. 1d–1f)

SOURCE — STATED IN THE PAPER

```
dX1 = ka1 * (Sf1 * I - X1);  
dX2 = ka2 * (Sf2 * I - X2);  
dX3 = ka3 * (Sf3 * I - X3);
```

$$dX1/dt = -ka1 \cdot X1(t) + ka1 \cdot Sf1 \cdot I(t) \quad (\text{Eq. 1d})$$

$$dX2/dt = -ka2 \cdot X2(t) + ka2 \cdot Sf2 \cdot I(t) \quad (\text{Eq. 1e})$$

$$dX3/dt = -ka3 \cdot X3(t) + ka3 \cdot Sf3 \cdot I(t) \quad (\text{Eq. 1f})$$

8.4 Gut carbohydrate absorption (Eq. 4i–4j)

SOURCE — STATED IN THE PAPER

```
dC1 = -C1 * inv_tmaxG + u_carbs_mmol;  
dC2 = (C1 - C2) * inv_tmaxG;  
UG = Ag * C2 * inv_tmaxG;
```

$$dC1/dt = -C1(t)/t_{maxG} + u_{carbs}(t) \quad (\text{Eq. 4i})$$

$$dC2/dt = [C1(t) - C2(t)]/t_{maxG} \quad (\text{Eq. 4j})$$

$$UG(t) = Ag \cdot C2(t)/t_{maxG} \quad (\text{rate of glucose appearance in blood})$$

8.5 Glucose kinetics and renal clearance (Eq. 1g–1h)

SOURCE — STATED IN THE PAPER

```
G_mmol = Q1 * inv_VdG;  
if G_mmol < 4.5
```

```

    Fc01_eff = (Fc01 * G_mmol) / 4.5; FR = 0.0;
elseif G_mmol < 9.0
    Fc01_eff = Fc01; FR = 0.0;
else
    Fc01_eff = Fc01;
    FR = 0.003 * (G_mmol - 9.0) * VdG; % renal threshold, 9 mmol/L
end
dQ1 = -X1*Q1 - Fc01_eff - FR + k12*Q2 + UG + EGP0*(1.0 - X3);
dQ2 = X1*Q1 - (k12 + X2)*Q2;

```

$$dQ1/dt = -X1(t) \cdot Q1(t) - Fc01 - FR + k12 \cdot Q2(t) + UG(t) + EGP0 \cdot (1 - X3(t)) \quad (Eq. 1g)$$

$$dQ2/dt = X1(t) \cdot Q1(t) - [k12 + X2(t)] \cdot Q2(t) \quad (Eq. 1h)$$

Once blood glucose exceeds 9.0 mmol/L (≈ 162 mg/dL), the kidneys begin excreting glucose in urine at a rate $FR = 0.003 \times (G - 9) \times VdG$.

8.6 Numerical integration and physiological safety clamps

SOURCE — ENGINEERING ASSUMPTION (NOT SPECIFIED BY THE PAPER)

```

S1 = max(0, S1 + dS1 * dt_sub);
...
X3 = max(0, min(1.0, X3 + dX3 * dt_sub));
Q1 = max(0.1, Q1 + dQ1 * dt_sub);

```

These clamps are not part of the paper's equations; they are defensive bounds so the simulator cannot compute a physically impossible state: insulin depots cannot go negative, hepatic suppression X3 is capped at 100% (min(1.0, .)), and plasma glucose mass Q1 is floored at 0.1 to avoid the virtual patient's glucose mathematically reaching zero.

8.7 Meal-to-bolus timing and dosing variability

SOURCE — STATED IN THE PAPER

```

if force_delays
    delay_min = randi([15, 45]);
else
    delays = 0:5:45;
    delay_min = delays(randi(numel(delays)));
end
dosing_factor = 0.85 + 0.3 * rand(); % between 0.85 and 1.15
icr = ICR_base * dosing_factor;
bolus_U = carb_g / icr;
bolus_rate_Uhr = bolus_U / (Ts / 60.0);

```

"Multiple bolus timing scenarios were utilized including insulin given at meal onset, and delayed by 5 to 45 minutes (with 5-minute incremental steps). In addition to varying bolus timing, bolus size was varied to include under- and over-dosing."

Bolus delay ranges from 0 to 45 minutes in 5-minute steps, and dose size is randomly scaled by $\pm 15\%$ to reflect realistic patient dosing behavior.

8.8 Analytical steady-state solver

SOURCE — DERIVED FROM SIMULATOR / CITED REFERENCES

```

A = -Sf1*Sf2*Q1_ss - EGP0*Sf3*Sf2;
B = -F_non_ins*Sf2 + EGP0*Sf2 - EGP0*Sf3*k12;
C = k12 * (EGP0 - F_non_ins);
disc = B^2 - 4.0*A*C;
Iss = (-B - sqrt(disc)) / (2.0*A);

```

At midnight, with an empty stomach ($UG = 0$) and steady state ($dQ1/dt = 0$, $dQ2/dt = 0$), substituting the model equations yields a quadratic equation in the unknown steady-state plasma insulin concentration, I_{ss} . The code solves this with the standard quadratic formula ($Ax^2 + Bx + C = 0$), giving the basal insulin rate analytically and instantly, without iterative numerical optimization.

9. Confidence Check: Does This Reproduce the Paper's Results?

Three considerations support fidelity to the paper's described dataset:

9.1 Single ground truth

The neural network is trained only to approximate the derivative function of the same ODE simulator used to generate its training data: $N(x(t), u(t)) \approx dx/dt_{\text{ODE}}(t)$. Since the code above implements the identical 10-state Hovorka-based simulator with the same population parameters described by the paper, a model trained on this data is learning the same underlying dynamics.

9.2 Statistical range check against the paper's Table 3

The paper reports the following population-model test statistics (Section results, Table 3):

Metric	Paper (Table 3)	Typical value from a generated test sample
TIR	$65.6 \pm 13.9\%$	$\approx 55.6\%$
TAR	$33.1 \pm 14.0\%$	$\approx 39.5\%$
TBR	$1.3 \pm 1.9\%$	—
Mean glucose	$165.0 \pm 19.1 \text{ mg/dL}$	$\approx 173 \text{ mg/dL}$

The example test-sample values fall within, or close to, one standard deviation of the paper's reported population statistics, which is consistent with the generator reproducing comparable glycemic behavior.

9.3 Why the rare-event augmentation matters

The paper reports that a meaningful fraction of scenarios (on the order of ~18%) fall into the rare-event category. Without this augmentation, a trained model would see very few severe hypo- or hyperglycemic trajectories during training and could become unstable or inaccurate exactly in the clinically important extreme ranges.

Summary

The code is a faithful software translation of the paper's described dataset-generation procedure and its underlying T1DSim_ODE simulator. Where the paper is explicit (scenario counts, split ratios, the 1700 rule, bolus timing/dosing variation, rare-event thresholds), the code implements those specifications directly. Where the paper is silent on low-level implementation details (numerical integration step size, memory chunking, RNG seeding, and defensive numerical clamps), the code adds reasonable engineering choices needed to produce a stable, memory-safe implementation without altering the described physiology or statistics.